Phoenix: Thermal-Aware On-Device Inference of Multi-Instance DNNs for Mobile Video Applications

SEUNGHYEOK JEON, Computer Science and Engineering, Yonsei University, Seodaemun-gu, Korea (the Republic of)
JIWON KIM, Uppsala University, Uppsala, Sweden
JEHO LEE, Computer Science and Engineering, Yonsei University, Seodaemun-gu, Korea (the Republic of)
HOJUNG CHA, Computer Science and Engineering, Yonsei University, Seodaemun-gu, Korea (the Republic of)

Running multiple deep neural networks (DNNs) simultaneously on mobile devices introduces challenges due to constrained computing resources. Previous research has explored the use of heterogeneous processors for accelerating DNN inference but often overlooks thermal issues, which can degrade computing power. In this article, we propose Phoenix, a system specifically designed to enhance the performance of multi-instance DNNs in video applications by maximizing accuracy and ensuring the achievement of a required frame rate. Phoenix allocates DNN tasks to the most suitable hardware processors, understanding complex thermal dynamics through reinforcement learning, and postpones the onset of thermal throttling. Despite optimized task allocation, continuous inference of multiple DNNs can still lead to thermal throttling. To manage performance degradation, Phoenix employs a multi-exit network, adaptively executing inference tasks to ensure consistent frame rates. Phoenix minimizes accuracy loss from early exits by optimally generating and operating multi-exit networks. We evaluated Phoenix using two different benchmarks and Virtual Youtuber streaming application. The results demonstrated that Phoenix effectively enhances device performance by delaying thermal throttling and achieving optimal accuracy while maintaining a consistent frame rate.

CCS Concepts: • **Human-centered computing → Ubiquitous and mobile computing;** • **Computer systems organization → Real-time systems**;

Additional Key Words and Phrases: Multi-exit network, multi-DNN inference, thermal throttling, heterogeneous processor

This work was partly supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2018-II180532, 50%) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00344323, 50%).
Authors' Contact Information: Seunghyeok Jeon, Computer Science and Engineering, Yonsei University, Seodaemun-gu, Seoul, Korea (the Republic of); e-mail: sh.jeon@yonsei.ac.kr; Jiwon Kim, Uppsala University, Uppsala, Uppsala County, Sweden; e-mail: jiwon.kim@angstrom.uu.se; Jeho Lee, Computer Science and Engineering, Yonsei University, Seodaemun-gu, Seoul, Korea (the Republic of); e-mail: jeholee@yonsei.ac.kr; Hojung Cha (corresponding author), Computer Science and Engineering, Yonsei University, Seodaemun-gu, Seoul, Korea (the Republic of); e-mail: hjcha@yonsei.ac.kr.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).
ACM 1539-9087/2026/03-ART31
https://doi.org/10.1145/3793860

ACM Reference Format:
Seunghyeok Jeon, Jiwon Kim, Jeho Lee, and Hojung Cha. 2026. Phoenix: Thermal-Aware On-Device Inference of Multi-Instance DNNs for Mobile Video Applications. *ACM Trans. Embedd. Comput. Syst.* 25, 2, Article 31 (March 2026), 23 pages. https://doi.org/10.1145/3793860

1 Introduction

On-device **artificial intelligence** (**AI**) is gaining popularity for its benefits in privacy, real-time responses, and reduced cloud costs. Notably, many emerging immersive applications such as **virtual YouTubers** (**VTuber**) [1] and virtual and augmented reality [2–4] require the execution of multi-instance deep neural networks (DNNs) within a single app. However, the limited computing resources of mobile devices make running such computationally intensive tasks challenging.

The use of heterogeneous mobile processors offers a viable solution for accelerating multi-instance DNN inference by running tasks across different processors to reduce overall latency. The key to this approach lies in matching each DNN task with the best-fit processor, considering the diverse specifications of processors and DNNs. Many studies have shown that leveraging heterogeneous mobile processors could speed up inference [5–7]; however, heat generation poses practical challenges. Mobile device performance is highly influenced by dynamic thermal conditions [8–10], yet there has been limited focus on addressing thermal issues in multi-instance DNN operations. Furthermore, many video applications require a specific frame rate for a smooth user experience, making it crucial to manage thermal challenges to maintain consistent performance.

In long-running, frame-driven video pipelines, thermal throttling introduces two key challenges that must be addressed to ensure a smooth and consistent user experience in multi-instance DNN workloads on mobile devices. First, when DNN tasks are assigned without considering the thermal sensitivities of heterogeneous processors, thermally weaker processors can overheat and throttle earlier. This not only shortens the high-performance period but also accelerates frame drops and latency spikes. Our preliminary results (Section 2.2) show that the point at which FPS violations begin can differ by up to 2 × depending solely on the task allocation policy—even under identical workloads. Second, even with thermally optimized task assignment, sustained execution of multiple DNNs inevitably produces excessive heat that exceeds the device's **Thermal Design Power** (**TDP**) [11]. Once thermal throttling begins, computing power drops non-uniformly across processors, as the timing and severity of slowdown vary depending on each processor's architecture and thermal inertia. Without an adaptive strategy that responds to these changes in real time, the system fails to maintain target frame rates, degrading user experience.

In this article, we propose Phoenix, a thermal-aware multi-instance inference system that co-optimizes task placement across heterogeneous processors and adaptive early-exit inference, to maintain sustained real-time performance under thermal throttling. First, Phoenix learns a thermal-aware task allocation policy using **reinforcement learning** (**RL**) [12, 13]. By observing how different configurations affect heat generation and overall latency, the RL agent learns to map each DNN task to the most suitable processor to postpone throttling as much as possible. Second, Phoenix employs a multi-exit network [14, 15], which includes several early exits along the depth of the network. By analyzing the slowdown patterns caused by thermal throttling, Phoenix configures an optimal set of exit points to maximize accuracy while maintaining stable inference performance under thermal fluctuations. Lastly, to achieve optimal accuracy while meeting real-time constraints, Phoenix offers two types of runtime schedulers: a best-effort scheduler that selects the most accurate exit based on the current operating condition, and a sustained performance scheduler that proactively adjusts exit depth to maintain stable performance over prolonged execution under thermal stress. The main contributions of this work are summarized as follows:

Table 1. Execution configuration of each task

DNN tasks	Config. 1	Config. 2
Face Detection	CPU (8)	NNAPI (SS)
Face Mesh	CPU (8)	CPU (4)
Face Recognition	NNAPI (FSA)	NNAPI (LP)
3D Pose Estimation	GPU	GPU

—To the best of our knowledge, Phoenix is the first thermal-aware on-device multi-DNNs inference system, specifically targeting video applications, designed for heterogeneous mobile processors.
—Introducing an RL-based DNN task allocation method, we effectively delay thermal throttling and improve long-horizon sustained real-time performance while meeting latency requirements.
—We design a thermal-aware multi-exit architecture by selecting exit locations based on processor-specific throttling-induced slowdown patterns.
—We propose lightweight runtime exit schedulers that adapt exit depth online to maximize performance under thermal throttling.

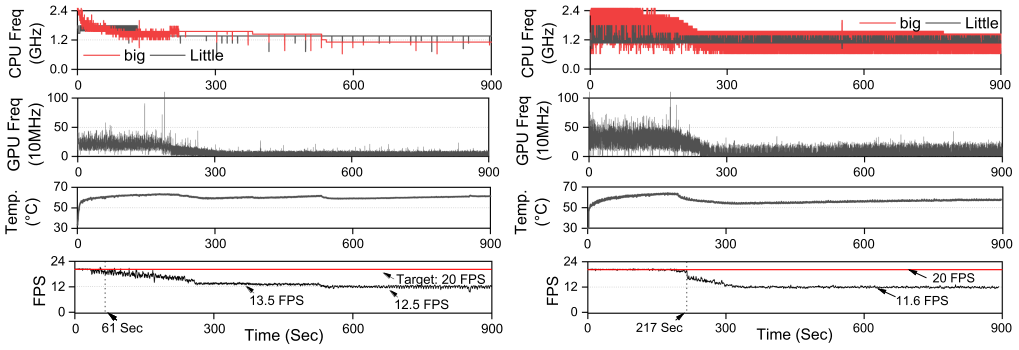
2 Motivation

2.1 Performance Effects on Task Allocation

Previous studies have demonstrated that running DNNs on mobile devices can induce thermal throttling [10, 16, 17]. However, there has been limited exploration into the influence of execution configurations on heat generation. To investigate the performance degradation patterns depending on execution configurations, we evaluated the Samsung Galaxy S22 running four different DNNs listed in Table 1. Each DNN task can be explicitly assigned to a specific processor, such as the CPU, GPU, or other accelerator like DSPs. When running on the CPU, the number of threads can be specified. For the accelerators, preference options such as **FastSingleAnswer** (**FSA**), **SustainedSpeed** (**SS**) or **LowPower** (**LP**) are selectable through NNAPI delegation [18]. We allocate each DNN task according to two different allocation policies as shown in Table 1. We then log the application's FPS, the device's temperature, and the clock frequencies of both CPU and GPU. The latency constraints are set to 50 milliseconds to ensure the application should run at 20 FPS.

Figure 1 shows the experiment results from two different policies. As illustrated in Figure 1(a), with Config. 1, the CPU frequencies start to decrease immediately after the application starts running, and the GPU frequency starts to drop at about 180 seconds. Due to this performance degradation, the application fails to meet the given latency constraints starting from 61 seconds. Following this, from approximately 280 seconds, it operates at about 13.5 FPS, and then, following a further drop in the CPU's big core frequency at 520 seconds, it operates at around 12.5 FPS. In the case of Config. 2, thermal throttling on the CPU occurs after 150 seconds, and the GPU after 200 seconds. As the GPU's performance degrades, the frame rate drops below 20 FPS and fails to meet the latency constraints. The frame rate decreases during the decline of GPU frequency, and eventually converges to 11.6 FPS at 310 seconds.

From the experiment results, we can draw the following insights. Allocating each task to the most suitable processor helps delay thermal throttling and maintain system performance. By dynamically adjusting task allocation based on the current thermal condition, it is possible to minimize latency violations while extending stable operation. Still, continuous DNN inference increases heat accumulation, which can eventually lead to thermal throttling. This underscores the



(a) Result from Config. 1

(b) Result from Config. 2

Fig. 1. Impact of different execution configuration on heat generation

need for strategies that maintain reliable performance even as thermal throttling impacts computing capacity.

2.2 Slowdown Pattern of Mobile Processor

Mobile processors comprise various types, each with distinct performance characteristics [19, 20]. We analyzed how performance degradation varies among each processor under thermal throttling conditions. We ran the application for 15 minutes using Config. 2 as described in Section 2.1 and logged the execution time of each DNN. Figure 2 presents a graph showing the distribution of the slowdown during the experiment for each processor. To compare the slowdown of each processor, we normalize the execution latency by dividing the execution time by the time without thermal throttling. A value of 1 represents the optimal case, while a value of 2 indicates the execution time has doubled. This graph provides two key insights. First, there exists a predominant slowdown range for each processor. For instance, the GPU, operating at frequencies of 640 MHz and 285 MHz, experiences a latency increase by approximately 1.2 to 2.3 times. This implies that each DVFS-enabled processor operates at certain frequencies most of the time. Second, the degree of slowdown varies across processors. This variation is due to each processor being designed for different purposes and having different frequency ranges. Thus, to provide a seamless user experience using heterogeneous mobile processors, we should carefully consider the different slowdown patterns of each processor.

2.3 Multi-Exit Network Inference

A multi-exit network is a model that incorporates multiple early exits at different depths, leveraging the tradeoff between DNN task accuracy and computational cost to mitigate latency violations. This characteristic makes it a promising option for addressing thermal issues in on-device DNN inference. However, existing research has rarely explored how the early exiting mechanism can be utilized to manage thermal throttling. To investigate this, we examined how multi-exit networks help delay thermal throttling and manage its effects when it occurs. We allocated DNN tasks as Config. 2, inserted two early exits into each DNN, and performed continuous DNN inference. Figure 3 shows the FPS traces over 15 minutes using different exits, where '#0' denotes an exit point in the backbone network, and higher exit numbers indicate shallower exits.

As shown in Figure 3, using a shallower exit significantly delays the onset of thermal throttling. When executing inference with exit #0, FPS starts to drop at 218 seconds, whereas #1 delays this drop until 392 seconds, and #2 maintains a stable FPS throughout the entire period. Additionally,

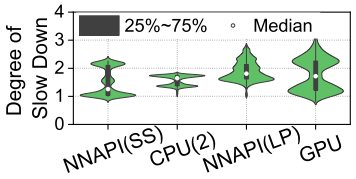


Fig. 2. Degree of slowdown on each processor.

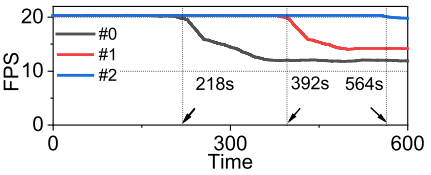


Fig. 3. FPS traces using different exits.

after thermal throttling sets in, selecting a shallower exit helps sustain FPS performance. For example, exit #0 experiences an FPS drop to 11.6, while exit #1 maintains at least 13.5 FPS, and exit #2 consistently holds 20 FPS without degradation. However, opting for an earlier exit inevitably leads to a reduction in accuracy. Therefore, an execution policy is required to balance accuracy and computational efficiency, ensuring optimal performance of DNNs.

3 Phoenix Overview

3.1 Problem Definition

DNNs inference task, which is composed of k different networks, on a mobile device. For each network, there are n different execution configuration options available. It can be represented as follows:

$$DNNs = \{dnn_1, dnn_2, \ \ldots, \ dnn_k\}, \tag{1}$$

$$Configs = \{config_1, config_2, \ \ldots, \ config_n\}, \tag{2}$$

where dnn_i means the ith DNN inference task and $config_j$ the jth execution configuration option, which includes both the processor, and the parameters used in execution. Here, each DNN has a different computational complexity, and the impact of thermals varies depending on the configuration used for each DNN's inference.

The objective of our work is defined in terms of two aspects: latency constraints and an accuracy goal. First, regarding latency, the overall execution time of DNNs inference, denoted as L_{app}, should meet the given latency constraints L_{cons} of the application. Here, we assume that the developers set the latency constraints of the application. Second, in the case of the accuracy, the overall accuracy of the application could be defined as follows:

$$Accuracy = \frac{1}{t_{end}} \int_{t=0}^{t_{end}} \left(\sum_{i=1}^{k} acc_i \right) dt, \tag{3}$$

where acc_i represents accuracy of the ith task, t denotes the time, and t_{end} the end time of the application. To sum up, our goal is obtaining the highest possible accuracy while satisfying latency constraints amidst thermal issues. In this article, we address this issue by allocating each DNN to the most suitable processor and configuring a multi-exit network to mitigate the performance degradation from thermal throttling.

3.2 Phoenix

Figure 4 provides an overview of Phoenix, a thermal-aware multi-instance DNN inference system for mobile devices designed to sustain high performance under dynamic thermal conditions. Phoenix first learns an optimal task allocation policy for heterogeneous mobile processors, aiming to delay thermal throttling while satisfying latency constraints. Even with the optimal allocation, continuous execution of DNNs inevitably triggers thermal throttling, leading to frame drops. To

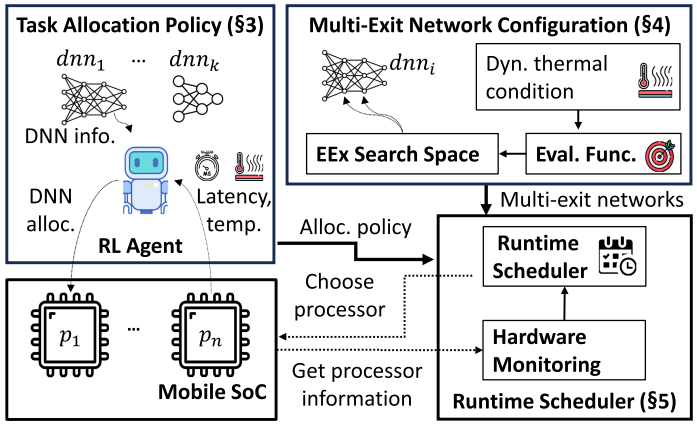


Fig. 4. Phoenix overview.

address this, Phoenix integrates a multi-exit network that dynamically selects an exit point at runtime, balancing computational load and accuracy. To minimize accuracy degradation from early exiting, Phoenix leverages NAS to configure multi-exit architectures optimized for processor-specific slowdown patterns. Finally, Phoenix's runtime scheduler orchestrates execution by selecting appropriate processor-task mappings and exit points, enabling efficient operation of multi-instance DNNs under thermal constraints.

4 Task Allocation Policy

4.1 RL-Based Task Allocation

Phoenix system first assigns k different DNN inference tasks to n different execution options. As observed in the experiments in Section 2.1, the execution time for multi-DNNs inference and the onset of thermal throttling vary depending on the execution configuration of each DNN.

The Need for DQN-based RL. Many algorithms are candidates for constructing the best task allocation policy. One could hand-craft a rule set to select execution options for multi-DNN inference. However, in practice, such rules are brittle for multi-DNN workloads: the set of concurrently running DNNs can vary across applications. As a result, the scheduler must account for a large number of task-combination cases and their shared-resource interference. A brute-force approach, which tests all task-execution combinations, is one option, but it becomes impractical as the search space grows exponentially with n^k. Alternatively, a simple workload-driven heuristic—such as assigning heavier tasks to more powerful hardware processors—can provide a practical baseline, but it cannot guarantee optimality across diverse task combinations and device conditions. More importantly, such heuristics cannot explicitly account for long-term thermal effects.

A learning-based approach provides a more robust solution than hand-crafted rules, because it can adapt to unknown device conditions and generalize across diverse multi-DNN combinations. Surrogate-guided supervised learning is possible, but it hinges on accurate label/oracle generation under delayed, coupled thermal effects. In particular, RL is well-suited to sequential decision-making problems [12, 13] where each action must be chosen based on the current device state while accounting for long-term consequences such as thermal accumulation and delayed throttling. In our setting, the scheduler observes runtime states (e.g., performance slowdown trends and thermal headroom) and selects an inference configuration for each DNN; it then improves its policy by receiving reward feedback that reflects both performance and thermal outcomes.

To evaluate the value of an action, RL methods range from tabular Q-learning [21, 22] to incorporating DNNs [23, 24]. Tabular Q-learning stores Q-values for each discretized state–action pair and can be effective when the environment can be enumerated reliably. However, Phoenix runs in dynamic mobile settings where runtime signals and multi-DNN interference vary over time, so the scheduler often faces unseen states. In such settings, the key requirement is to approximate $Q(s, a)$ in a way that generalizes, especially near throttling/FPS-violation boundaries with delayed thermal effects. Coarse discretization may miss boundary effects, whereas fine discretization is hard to reuse across conditions. Phoenix therefore uses a DQN to approximate $Q(s, a)$, allowing the policy to learn nonlinear signal–outcome relationships and generalize beyond visited states.

Method Overview. Phoenix learns task-allocation policies offline and applies them with minimal overhead at runtime. During training, each DNN is managed by a dedicated DQN agent. The agent observes device-level signals that capture shared-resource contention and thermal conditions, and updates its policy from collected transition tuples. To avoid the constant overhead of online DQN inference, Phoenix deploys a lightweight rule table derived from the learned policy. The table is indexed by the CPU/GPU operating-frequency pair and enables fast, predictable runtime selection of each DNN's execution configuration. Note that the deployed rule table is not a Q-table over the full RL state. Instead, it is a compact, deployment-oriented lookup keyed only by low-overhead runtime signals, namely the selected CPU/GPU frequency pair, which allows deterministic and constant-time decisions. In contrast, the RL training state includes continuous, high-cardinality signals such as per-processor slowdown trends, thermal headroom, and contention indicators across concurrently running DNNs. Building a classic Q-table for $Q(s, a)$ would therefore require discretizing multiple continuous dimensions and their cross-products, yielding either an impractically large table or overly coarse bins. Coarse bins are particularly harmful around throttling and FPS-violation boundaries, where small changes in headroom or slowdown can flip the correct action, while fine bins lead to sparse visitation and poor reuse when the active DNN set and interference patterns change.

4.2 Defining State, Action, and Reward

Obtaining an optimal policy requires carefully designing the states, actions, and reward functions with consideration of thermal dynamics. In this section, we define these RL components to generate an allocation policy that proactively mitigates thermal buildup and delays throttling.

State. The state should comprehensively capture both the current thermal conditions and computational demands, providing the foundation for informed action selection by the agents. Phoenix manages each DNN with a dedicated RL agent, but these agents are not independent. They observe a shared global state vector that encodes device-level signals reflecting cross-DNN resource contention, such as global temperature, processor frequency and utilization, and slowdown proxies. We first include shared information about each mobile processor. Phoenix collects the utilization, operational frequency, and current temperature of the CPU and GPU through the Android framework, normalizes these values against their maximums, and adds them to the state representation. For other accelerators, obtaining such detailed statistics at runtime is challenging, so we use a proxy value that represents the degree of slowdown for each accelerator, defined as

$$Freq_{processor} = \frac{L_{now}}{L_{normal}}, \tag{4}$$

where L_{now} is the current execution time using the specified processor, and L_{normal} is the latency under normal conditions without thermal throttling.

In addition to these shared device-level features, each agent also receives task-specific descriptors, including the static FLOPs of its DNN to indicate backbone heaviness. Although FLOPs

is constant, configuration- and thermal-driven demand changes are captured by the shared latency/slowdown signals, and static descriptors are often sufficient to condition the policy on task identity and relative heaviness.

Action. The action space of the RL system should include all possible configurations for DNN inference execution. For the CPU, we configure the number of threads for DNN inference. When using accelerators with NNAPI, we can provide execution options such as *FastSingleAnswer, LowPower*, or *SustainedSpeed.* Additionally, we have made it possible to directly use the GPU by employing a GPU delegation. The configured action space is as follows:

$$P = \begin{cases} CPU(number\ Of Threads) \\ GPU \\ NNAPI(acceleratorOption) \end{cases}, \tag{5}$$

Reward Function. The reward function of Phoenix is constructed with two primary goals: latency and heat generation. When failing to meet the latency requirements, the penalty is adjusted based on the extent to which the latency exceeds the target latency as follows:

$$Penalty_{latency} = \begin{cases} \alpha(L_{now} - L_{app}) & L_{now} > L_{app} \\ 0 & L_{now} \leq L_{app} \end{cases}, \tag{6}$$

where α is the weight for latency violation.

Since heat generation is another goal for the RL agent, we impose a large penalty when thermal throttling occurs. To encourage the RL agent to learn policy that postpones thermal throttling, we design the penalty to decrease over time, as follows:

$$Penalty_{thermal} = -\frac{n_{large}}{t_{tt}}, \tag{7}$$

where t_{tt} denotes the amount of time till thermal throttling occurs, and n_{large} is a large number. We set n_{large} to 900 in Equation (7) to impose a strong penalty on early thermal throttling.

Here, offering a large-step reward, which occurs infrequently, can impede the RL agent's ability to learn the policy effectively. Taking this into account, we designed the reward function to include temperature changes in the CPU and GPU at every step. The greater the temperature change, the larger the penalty applied. The final formula is structured as follows:

$$Reward = Penalty_{latency} + Penalty_{thermal} - \Delta Temperature_{cpu} - \alpha\ \Delta Temperature_{gpu}\ , \tag{8}$$

where $Temperature_{cpu}$ denotes the temperature of the CPU, $Temperature_{gpu}$ the temperature of the GPU, and α is a weighting factor greater than or equal to 1, used to impose a larger penalty for GPU temperature variations. We introduce α because the GPU plays a more critical role in sustaining inference performance compared with the CPU. Thus, controlling GPU temperature more aggressively is essential for maintaining overall system efficiency. This reward is computed at the application level and shared by all RL agents. When any agent chooses an aggressive configuration that violates the latency target or accelerates thermal buildup, the resulting penalties are reflected in this common reward, which all agents receive. As a result, the agents are naturally encouraged to behave cooperatively and learn allocation policies that jointly satisfy the latency targets while postponing thermal throttling.

4.3 Learning and Deployment

Although Phoenix maintains a separate RL agent for each DNN, these agents do not optimize independent objectives. All agents observe the same global state and receive the same scalar reward defined above. When one agent selects an aggressive configuration that increases latency or accelerates thermal buildup, the resulting slowdown and temperature rise are reflected in the shared

ALGORITHM 1: Learning and Deployment

```
1:  def Learning (memory array M[ ], task DNN):
2:   while policyConverge:        // until policy convergence
3:    waitForLongTime()           // cool down the device
4:    while singleEpisode:
5:     for each dnn_i in DNN:
6:      action = argmax(DQN(observeState()))
7:      action = action if rand() > ϵ else randomAction()
8:      setHardwareConfig(dnn_i, action)
9:      latency, Δheat = executeInference(numInference)
10:     store(state_curr, action, state_next, reward(latency, Δheat) to M_i
11:     updateDQNIfMemoryFull()
12: def Deploying (DQN, task DNN)
13:  waitForLongTime()       // cool down the device
14:  while singleEpisode:
15:   freqPair = observeFreqOfEachProcessor()
16:   for each dnn_i in DNN:
17:    actionVector[i] = argmax(DQN(observeState()))
18:    setHardwareConfig(dnn_i, actionVector[i])
19:   executeInference(numInference)
20:   actionDict[freqPair].append(actionVector)
21:  actionDict = filterPrimaryKeys(actionDict)
22:  return actionDict
```

state and reduce the global reward. During training, this feedback discourages selfish actions that harm the overall system and guides all agents toward a policy that jointly satisfies the latency target while postponing thermal throttling. Here, Algorithm 1 describes the specific methods Phoenix uses to construct and deploy allocation policies.

Learning. The goal of RL is to maximize the sum of rewards within a single episode. In our problem, we define one episode as the execution of an application on a device. To accurately analyze the impact of execution configuration on heat generation, the device is allowed to cool down sufficiently at least 30 minutes before executing each episode (line 3). After the episode starts, the application is launched and continuously computes DNN inference task (line 4). For the inference, the agent observes the current state and selects the optimal action, occasionally choosing a random action with a small probability (lines 6–7). The reason for random selection is to make RL agent explore the unknown policy. After deciding the inference configuration of each DNN, the application computes DNN inference for a specified number of times then calculates the reward by observing the average execution time, thermal throttling, and heat generation (line 9). The updated state is recalculated and the tuple of (S, A, S', R) is stored in memory (line 10). If enough experience data has been collected, then the agent update the policy by training the deep neural network of the DQN agent (line 11). Lastly, Phoenix repeats this process until the sum of the rewards within a single episode converges (line 2).

Deploying. Running the DQN model at runtime for obtaining the optimal configurations requires additional computing overhead. To mitigate this overhead, we deploy the learned policy by converting it into a rule-based algorithm. We simplify the problem by finding the optimal execution configuration under the constraint of each processor's maximum capacity. Here, the maximum capacity can be reduced by thermal throttling. To obtain the optimal rule, we executed the DNN inference using the optimal configuration suggested by DQN, starting from normal conditions without thermal throttling. By profiling the frequency of mobile processors and chosen actions,

we create rules for the optimal configuration corresponding to the frequency pairs of the CPU and GPU. The "Deploying" function in Algorithm 1 describes the detailed process for this conversion.

We cool down the device for a sufficiently long period again (line 13). Before taking action, we monitor the current operating frequencies of the CPU and GPU (line 15). Then, using the trained DQN, we select only the best actions—differently from the learning phase—and adjust the execution configuration accordingly (lines 17–19). After collecting the data, we construct a rule based on the data. Here, to reduce the number of rules, $filterPrimaryKeys$ function is utilized to extract top-k most frequently used keys until their cumulative usage reaches 95%. Phoenix ultimately deploys the remaining rules on the mobile device (lines 20–22).

5 Multi-Exit Network Configuration

5.1 Need of Optimal Early-Exit Architecture

The task allocation method can minimize heat generation and delay thermal throttling; however, continuously executing DNNs inference can still cause throttling [10, 16, 17]. When thermal throttling occurs, device performance degrades, extending execution time of DNN inference and leading to a decrease in the FPS. Phoenix copes with the frame drop problem by employing multi-exit networks [14, 15, 25, 26] which balance accuracy and latency by adaptively determining exit points.

Specifically, the neural architecture of the early-exit determines the computational demand and accuracy while early-exiting [14]. Designing early-exit networks involves deciding the number of exits, placing each exit, and configuring the architecture of each exit. Experimental results from Section 2.2 show varying performance degradation patterns across mobile processors, emphasizing the need of careful consideration in exit design. To this end, we design **neural architecture search** (**NAS**) system [27, 28] considering the performance degradation pattern of each processor.

First, we define search space for early-exits based on the architectural characteristics of multi-exit network. Since each DNN has a unique architecture, Phoenix allows users to input a customized search space to accommodate the specific structures of their DNNs. Next, using a hardware-in-the-loop approach, we set the smartphone's CPU/GPU frequencies and measure the execution time of early-exits under thermal throttling conditions. Phoenix then identifies the optimal neural architecture that maximizes accuracy while adhering to latency constraints.

5.2 Search Space

The search space for early-exit networks must include the number, placement, and structure of early exits.

Number. Having too many early exits can potentially degrade the overall performance of the network by negatively impacting the convergence during training [14, 29]; hence, it is crucial to determine an adequate number of exits. Our preliminary experiments show that while mobile processors offer a wide range of frequencies, the frequency options used for extended periods are limited. Based on this observation, we configured the number of exits as the number of deployed rules. For example, if the "Deploying" function of Algorithm 1 outputs three different keys—frequency pairs for CPU and GPU—each network can have a maximum of three exits. In some cases, networks may meet the target latency without needing early exits, and such scenarios are not included in the count of early exit networks.

Location and Early-exit Architecture. Recent neural networks consist of multiple consecutive blocks, with each block containing several layers. We configured the search space to allow early exits to be selectively positioned after any block. However, running the backbone network under thermal throttling conditions may result in failure to meet the given latency requirements if inference proceeds beyond a certain block. Based on this, we excluded blocks that do not meet latency criteria from the search space when determining exit locations.

Meanwhile, since each network has a different structure, defining a general search space for exit architectures is challenging. Phoenix offers two options to address this issue. First, Phoenix allows the use of the existing backbone network's exit architecture as a constant. Then, the search space only includes the number and location of early-exit networks. Second, we propose incorporating a customized search space for early-exit networks to further enrich the exploration. By utilizing a larger search space, Phoenix can construct a more effective early-exit architecture.

5.3 Evaluation Function and Searching

The objective function is calculated based on the fulfillment of specified latency requirements by the given early exits and their impact on accuracy. Measuring accuracy is straightforward using a validation dataset. To accurately obtain the execution time of the DNN inference task, we exploit the hardware-in-the-loop approach. After training the discovered early-exit network on a remote server, we deploy the model on a mobile device. Our NAS system then sets the CPU and GPU frequencies considering thermal throttling and measures the execution time.

The goal of NAS is to search for the optimal multi-exit architecture that achieves the highest accuracy while adhering to latency constraints under limited frequencies. However, when more than two DNNs run on specific processors, the execution of DNNs inference must be completed within L_{cons}. In this case, we distribute the latency budget based on the execution time proportion of each DNN's backbone network, allowing users to adjust this ratio as needed. The goal is then expressed as follows:

$$a_i^* = \underset{a_i \in A}{\operatorname{argmax}} \, Acc\,(\alpha_i) \; \mathbf{s.t.} \; L\,(a_i, freq) < L_{cons}\,(i) \;\; \& \; \Sigma_{\mathrm{k=1}}^{\mathrm{n}} L_{cons}\,(k) \leq L_{cons} \tag{9}$$

where a_i^* represents the optimal early-exit network for dnn_i, A the search space, $Acc(a_i)$ the accuracy of the discovered early exit a_i, $L(a_i, freq)$ the measured execution time of a_i under a specific frequency $freq$, and $\Sigma_{\mathrm{k=1}}^{\mathrm{n}} L_{cons}(k)$ the sum of latency budget for the DNNs run on the processor where dnn_i is executed. Our NAS system employs the aging evolution method to explore the search space.

6 Runtime Scheduler

The runtime scheduler is composed of two key components, task allocation and exit selection, each relying on outputs from Sections 4 and 5, respectively. The TaskAlloc function in Algorithm 2 demonstrates the overall task allocation process. Phoenix first calculates the CPU and GPU frequencies using a moving average to account for temporary frequency boosts due to touch interactions (line 2). It then determines the closest frequency set based on the Euclidean distance from the current frequencies (line 3) and assigns each DNN task accordingly (lines 4–5).

For exit selection, we introduce two scheduling strategies: best-effort and sustained performance. The best-effort method maximizes accuracy at each instance, as described in the BestEffort function in Algorithm 2. In an ideal scenario, the multi-exit network obtained would allow selecting the best exit based on the current operating frequency. However, in real-world, factors such as background applications and touch boosts can introduce fluctuations in DNN inference latency. To prevent such potential latency violations, Phoenix checks whether the execution latency of each DNN satisfies L_{app} at every inference cycle. If the measured execution time exceeds the latency constraints (lines 7–8), the scheduler enforces a shallower exit (line 9) to ensure real-time performance.

While effective in short bursts, the best-effort approach can lead to rapid thermal throttling in prolonged execution. To mitigate this, we introduce the sustained performance method, implemented in the SustainedSpeed function. This approach proactively applies early exiting before

ALGORITHM 2: Runtime Scheduler

```
1:  def TaskAlloc (memory M, task DNN):
2:    freq = movingAverageOfFreq()
3:    taskAlloc = argmin(freq − actionListkeys())
4:    for each dnn_i in DNN: // task allocation
5:      setHardwareConfig(dnn_i, taskAlloc[i])
6:  def BestEffort (task DNN):
7:    for dnn_i in DNN:
8:      if latencyOfCurrentExit(dnn_i) >L_app:   // if latency constraints violation occurs
9:        useEarlierExit(dnn_i)                  // move to an earlier exit (update current_exit[dnn_i])
10: def SustainedSpeed (task DNN, threshold)
11:   for dnn_i in DNN:
12:   temp, latency = getTempLatency(L_app, targetTemp)
13:   error_t, error_l = getError()
14:   control_t, control_l = pidControl(error_t, error_l)
15:   exitProb = sigmoid(control_t/threshold)
16:   if latency > L_app :
17:     useEarlierExit(dnn_i)
18:   else if temp > targetTemp & random() < exitProb :
19:     selectEarlierExit(control_t, threshold)
```

thermal throttling occurs, ensuring long-term performance stability. Using control theory, the algorithm continuously monitors real-time temperature and latency feedback (lines 12–13). Here, the target latency is set by the application's constraints, while the target temperature is determined by the device's thermal throttling threshold. Then, Phoenix computes control signals to adjust exit selection dynamically (lines 14). Instead of deterministically enforcing early exits when the temperature exceeds a threshold, the scheduler uses a probabilistic exit mechanism using sigmoid function (line 15). If latency exceeds L_{app}, early exit is immediately enforced (lines 16–17). Otherwise, if the system temperature surpasses the thermal threshold, the scheduler probabilistically selects an earlier exit based on the control signal (lines 18-19), ensuring a gradual and adaptive response to thermal variations.

7 Implementation

The RL method for task allocation policy and the runtime scheduler operates on the mobile device, while the neural architecture system is handled by the GPU server. First, we developed a demo application using the Unity engine for implementing task allocation policy generator and the runtime scheduler. This demo app uses TensorFlow Lite (TFLite) to execute DNNs on-device. Since the RL method is used only for constructing allocation policy, the code is executed only during the learning phase. For the runtime scheduler, the TFLite C++ Subgraph API was used to implement the adaptive inference mechanism of a multi-exit network.

Next, we implemented the NAS system on a remote server equipped with 8 NVIDIA GeForce RTX 3090 GPUs. For the NAS system, we adjust the width and depth of the early-exit networks within the search space. Since the exit architecture of each DNN is different, search space of each DNN should be configured differently. We configure the search space of each DNN by modifying the exit of the given network. We integrated the number of layers and filters into the search space for convolutional layers, as well as the number of layers and hidden nodes for fully connected layers. The search space for the number of filters and hidden nodes is determined to range from half to double of the original exit from the backbone network. We then configured the search space for the number of layers to allow for the addition or removal of two layers. After obtaining the

Table 2. NAS Hyperparameters

Category	Parameter	Value
Population & Sampling	Population size	50
	Sample size	10
Training budget	Total evaluated candidates	Total evaluated candidates: 400
	Per-candidate training	8 epochs
	Learning rate/Weight decay	3e-4, 1e-4



(a) VTuber (b) Smart lens

Fig. 5. Reward convergence in learning phase.

best architecture, we train the architecture from scratch using the knowledge distillation method and convert the PyTorch model to a TFLite model.

8 Evaluation

8.1 Experiment Setup

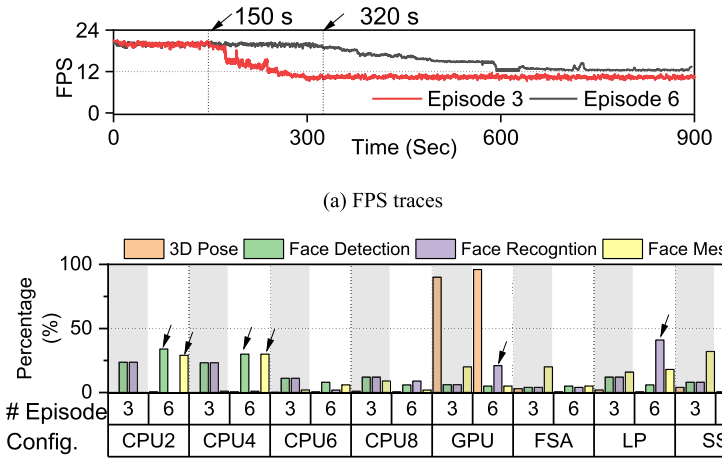
Scenarios. To demonstrate the effectiveness of the proposed system, we considered two scenarios. The first scenario involves a VTuber application [30], where virtual characters express the actions and emotions of a human streamer. For this scenario, we configured a setup where four different DNNs—3D pose estimation [31], face detection [32], face recognition [33], and face mesh [34]—operate simultaneously on-device. The second scenario envisions an application involving smart lens, where object detection [35] and object classification [36] are performed concurrently, and the user is provided with the classified objects.

Platform. We conducted experiments with Phoenix on three different mobile devices: Samsung Galaxy S22, Galaxy S25, and Google Pixel 7 Pro. We set both apps to run at 20 FPS on the Galaxy S22 and S25, while the Pixel 7 Pro is set to run at 15 FPS, considering its specifications. To control the temperature during task allocation policy generation, we used the SU-642 temperature chamber from ESPEC, and pre-set the temperature of the smartphones to 25 degrees Celsius before the experiments. Additionally, the duration of each episode was set to 15 minutes. Next, utilizing our configured searching system, we deployed networks optimized for each device. Lastly, we operated the runtime scheduler on each device to evaluate the performance of Phoenix.

NAS Setup. Table 2 summarizes the key constants used in our aging-evolution NAS pipeline, including the population and sampling configuration and the training budget for candidate evaluation

8.2 Efficacy of RL-Based Task Allocation

Phoenix begins with the RL agent learning the task allocation policy. Figure 5 shows the normalized sum of rewards for each episode while learning the policy across three smartphones and two scenarios. In both cases, at the beginning of the learning process, the DQN is untrained; thus, the



(a) FPS traces

(b) Chosen action

Fig. 6. Performance comparison of models trained on different episodes with VTuber (Galaxy S22).

RL agent selects actions randomly. The sum of the rewards is relatively low because the RL agent assigns each task to inappropriate processors, leading to early thermal throttling and a lower frame rate. In the VTuber case, as shown in Figure 5(a), convergence is observed across all types of smartphones around the 6th to 7th episodes. This indicates that the RL agent has developed a suitable task allocation policy for each situation through a process of trial and error. Next, in the smart lens scenario, as shown in Figure 5(b), the sum of rewards converged around the 4th to 5th episodes. The reason for the faster convergence in this case, compared with the VTuber case in Figure 5(b), is that the number of tasks was relatively small, making it easier to solve the allocation problem, unlike the VTuber case, where frequent task reallocations were necessary due to varying options

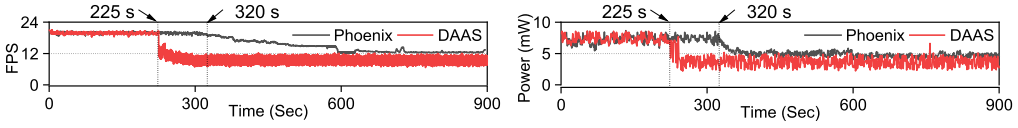
To further analyze the learning process, we conducted an experiment using two different DQN models on the Galaxy S22 with VTuber scenarios: one trained up to episode 3 and the other trained up to episode 6. Figure 6(a) shows the FPS traces. First, in the case of Episode 3, it is notable that the required FPS is initially met in most situations until 150 seconds. Thermal throttling occurs at 150 seconds, causing significant frame drops and a subsequent decline to 9.6 FPS. In the case of Episode 6, FPS meets the latency requirements of the application until 320 seconds. Thermal throttling occurs much later compared with Episode 6 and even after thermal throttling, the frame rate is higher than the case of Episode 3. This demonstrates a considerable improvement over learning phase.

Next, Figure 6(b) shows the percentage of action selected for each DNN during the experiment. For 3D pose estimation, execution primarily took place on the GPU in episodes 3 and 6, reducing latency violations by leveraging the GPU's powerful computing resources. In other tasks, we found that multiple inference configurations were used for DNN processing during the 15-minute experiment. Specifically, in episode 3, due to incomplete training, a wide variety of options are utilized for execution, whereas in episode 6, execution primarily relies on two or three options (Arrows indicate the configurations most frequently used in episode 6). The last observation is that, for Face Detection and Face Recognition, only two threads were initially used; however, over time, usage expanded to as many as four threads. To evaluate the efficacy of the proposed task allocation policy, we compared the proposed method with the DAAS [6]. DAAS is an RL-based technique for allocating multi-instance DNNs on a heterogeneous mobile processor taking into



(a) VTuber (b) Smart lens

Fig. 7. Distribution of FPS (Ours vs. DAAS).



(a) FPS traces (b) Power traces

Fig. 8. A comparison of the Phoenix with the prior work for task allocation method.

account low power consumption and execution. Notably, DAAS focuses on selecting execution options and does not incorporate early-exit inference; thus, it always executes the full backbone for each DNN. For a fair comparison, we evaluated only the task placement component of Phoenix by disabling early-exit inference, and compared the resulting placement decisions against DAAS. Figure 7 presents distribution of the FPS over 15 minutes using Phoenix and prior work. Phoenix is shown to outperform DAAS across all different smartphones and scenarios. We can observe that the VTuber scenario, which consists of relatively heavy tasks, showed greater benefits compared with the smart lens scenario. This is because performance of DAAS drops sharply after heat generation occurs. In the VTuber case, where more tasks are co-running, heat generation increases, causing thermal throttling to occur more quickly.

To compare the detailed operation of Phoenix and DAAS, we analyzed the VTuber scenario running on the Galaxy S22. Figure 8(a) shows the results from both Phoenix and DAAS. The FPS trace of DAAS meets the latency requirements for 225 seconds, but then experiences a significant drop in frame rate, which is notably shorter than the 320 seconds observed with Phoenix. Since existing research considered low power and latency in constructing allocation policies, the scheduler performs adequately but not optimally in an execution environment without thermal throttling. However, following the onset of thermal throttling, the FPS trace of DAAS fluctuates significantly, indicating that previous work did not adequately address the device's performance degradation under such conditions. In contrast, Phoenix indeed postpones the onset of thermal throttling and also performs well amid degraded computing conditions.

These results stem from DAAS, which failed to adequately address thermal throttling issues while assuming a consistent level of computing power. It imposes a constant penalty when latency constraints are not met. When thermal throttling occurs, and no action can satisfy the latency constraints, the RL agent of DAAS fails to learn an effective policy. Furthermore, prior work does not include the operating frequency of mobile processors in the state of the RL system, thus preventing the agent from accurately understanding the exact computing power of mobile processors.

8.3 Effectiveness of Discovered Architectures

Table 3 shows the performance of models found by Phoenix across different tasks. An exit of the given backbone network was marked with '#0' and a higher exit number denotes a shallower exit.

Table 3. Discovered Architecture # Exit Number (accuracy, Latency in msec)

Task	Galaxy S22	Galaxy S25	Pixel 7 Pro
Face Detection	#0: (–, 17)	#0: (–, 9)	#0: (–, 23)
Face Mesh (NME)	#0: (–, 7.8) #1: (1.99, 6.5)	#0: (–, 3.9)	#0: (–, 18) #1: (2.02, 12) #2: (2.55, 10)
Face Recognition (Top 1 Acc.)	#0: (99.03, 32) #1: (98.15, 22) #2: (95.45, 18)	#0: (99.14, 33)	#0: (99.02, 53) #1: (97.44, 43) #2: (94.78, 38)
3D Pose Estimation (mm MPJPE)	#0: (59, 34) #1: (66, 25) #2: (79, 16)	#0: (56, 36) #1: (60, 29)	#0: (60, 47) #1: (68, 38) #2: (89, 28)
Object Detection (mAP@0.5)	#0: (27, 37) #1: (23, 19)	#0: (29, 23) #1: (26, 19)	#0: (28, 48) #1: (23, 31) #2: (18, 24)
Object Identification	#0: (77, 35) #1: (72, 19)	#0: (79, 26) #1: (76, 20)	#0: (76, 52) #2: (70, 24)

Indicating accuracy with '–' means that the model cannot be evaluated because MediaPipe do not provide open dataset.

The latency is measured by running the DNN task on the CPU with four threads, except for the 3D pose estimation and object detection, which are executed solely on the GPU. As shown in the table, there are fewer early exit networks found for the Galaxy S25 compared with those for the Galaxy S22. This is because more powerful smartphones require fewer instances of failing to meet the target FPS. Additionally, the overall accuracy tends to increase with the latest smartphones. With the Galaxy S25, it showed the capability to operate multiple-instance DNNs at the same FPS with minimal accuracy loss. On the other hand, the Pixel 7 Pro showed the lowest accuracy, attributed to the weak computing power of the device. Thus, the early-exit networks are shallowly located and have simpler architecture.

Additionally, one notable point from Table 3 is that the goolgle-lens application requires fewer exits compared with the VTuber application. This is because, when running applications composed of relatively lightweight tasks, even if thermal throttling occurs, the resulting frame drops are comparatively less significant, allowing for sufficient service without frame drops with fewer exits. Furthermore, unlike the cases observed in the VTuber application, the additional exits in the smart lens application are inserted in relatively deeper layers, leading to a smaller accuracy loss.

To evaluate the efficacy of the model found by Phoenix, we implemented the NAS methodology proposed by previous work [37], excluding their search space. Instead, we utilized Phoenix's search space for the fair comparison. The existing NAS approach for constructing multi-exit networks running on smartphones sets the latency constraints of early-exit networks based on the highest and lowest frequencies of the given processor. Then, it divides the resulting latency intervals into several equal parts, thereby establishing the latency constraints for each exit.

We compared the best models obtained using two different NAS methodologies for 3D pose estimation on the Galaxy S22 device. For the existing work, we construct two different models with 3 exits and 5 exits. Here, as illustrated in Figure 6, the 3D pose estimation mostly operates on the GPU; hence, the execution time was measured by running DNN on the GPU. The solid line in Figure 9 shows the accuracy and latency of three different discovered models from Phoenix and previous work. In terms of accuracy, the model from Phoenix demonstrates the highest accuracy,

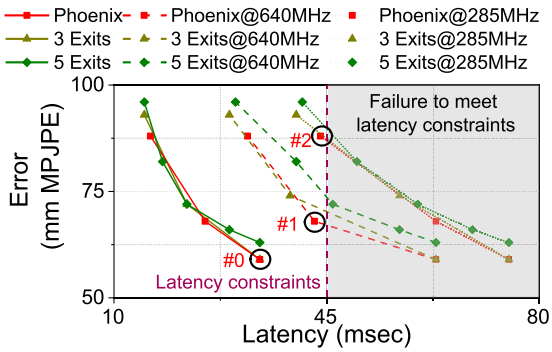


Fig. 9. Discovered 3D Pose Estimation models from Phoenix and previous work.

followed by the model from previous work found with three exits, and then by the model with five exits. This is because while training multi-exit networks, the loss function is defined in the form of a sum across all exits, and the use of unnecessarily many exits can degrade overall accuracy of the network. Phoenix, on the other hand, analyzes the performance degradation pattern due to thermal throttling and determines the number of exits and target latency accordingly. Therefore, Phoenix can outperform models with arbitrarily chosen numbers of exits.

Next, as observed in the preliminary experiment, since the GPU of the Galaxy S22 operates dominantly at 640 MHz and 285 MHz, we set the GPU of the mobile device to these frequencies and measured the latency. The experiment results are illustrated with the dashed and dotted line in Figure 9. The first early exit network found by Phoenix operates at 43 milliseconds at 640MHz, and the second early exit network operates at 44 milliseconds at 280 MHz. This shows that the early exit network architecture Phoenix discovered could meet the latency constraints during thermal throttling.

For the model with three exits, it was observed that the first early exit meets the latency constraints at 640 MHz; however, its accuracy is significantly lower than that of the first early exit in Phoenix, resulting in significantly poorer performance. In the case of network with five exits, despite the model having an exit suitable for each specific frequency, it consistently underperforms compared with the model discovered by Phoenix.

8.4 Performance of Runtime Scheduler

Testbed. We implemented a VTuber application to demonstrate the proposed Phoenix can be practically used for real-world applications. VTuber is an application where virtual characters express the actions and emotions of the human streamer. According to Business Research Insight, the VTuber market is expected to grow at a CAGR of 70%, reaching USD 498 billion by 2032, indicating a market with significant potential. We developed a Unity application, as shown in Figure 10, that executes four different DNNs on-device, as previously mentioned in the VTuber scenario. The program captures camera frames from a smartphone to estimate the 3D pose of a person, recognize their face, identify who they are, and estimate their facial landmarks. Using the computational results from each DNN, we control Unity's 3D human model to achieve the implementation.

Performance of Runtime Scheduler. We then evaluated the overall performance, i.e., FPS and accuracy, of Phoenix on real mobile devices using two different inference strategies proposed in Section 6. Table 4 shows the FPS and expected accuracy when running multi-instance DNNs on different smartphones using two different methods. BE represents the best-effort method, while

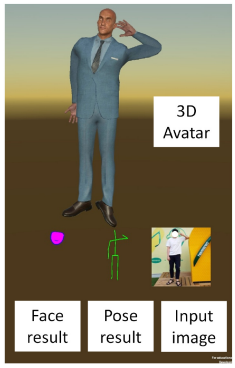
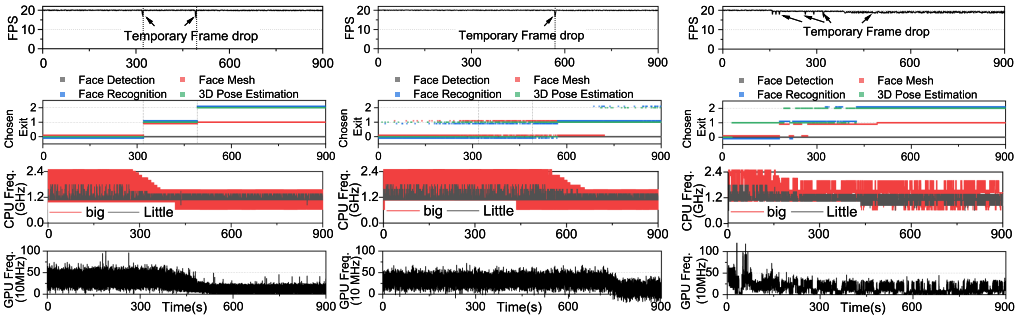


Fig. 10. VTuber application.

Table 4. Performance of Runtime Scheduler.

Task	Galaxy S22		Galaxy S25		Pixel 7 Pro	
	BE	SS	BE	SS	BE	SS
Mean FPS	19.97	19.97	19.99	19.99	14.87	14.98
Face Mesh (NME)	1.99	1.99	–	–	2.44	2.15
Face Recog. (%)	97.23	98.14	98.62	98.91	96.21	97.21
3D Pose Est. (MPJPE)	69.41	67.25	63.39	60.23	76.62	72.39

Indicating accuracy with '-' means that the model cannot be evaluated because MediaPipe do not provide open dataset.



(a) Best-effort method (b) Sustained speed method (c) Best-effort method w\ screen recorder

Fig. 11. Operations of VTuber application powered by Phoenix on Galaxy S22.

SS refers to the sustained speed method, where inference was performed for 15 minutes. As seen in the table, the average FPS across all smartphones is close to the target FPS. The discrepancy between the target and actual average FPS is primarily due to the non-DNN inference components of the application. Since no early-exit network was applied to face detection, its accuracy has been excluded from the table. The experimental results indicate that, overall, the sustained speed method achieves higher accuracy compared with the best-effort approach. This is because sustained speed utilizes intermediate early exits before thermal throttling occurs, effectively delaying its onset. As a result, the frequency of using the shallowest exit, which causes the most accuracy loss, is relatively lower when using the sustained speed method, leading to improved overall accuracy.

We then further analyze the performance of the two different inference methods using a Galaxy S22 smartphone. The experiment is conducted by running only the Unity application without any background applications or touch interactions. Figure 11(a) presents the FPS, chosen exit, and the CPU and GPU frequencies when using the best-effort method. As shown in the figure, similar to the previous table, the FPS nearly meets the target FPS in almost all cases. From the start until approximately 320 seconds, all DNN tasks select Exit 0, achieving the best possible accuracy. From around 320 to 500 seconds, the system transitions to Exit 1, and towards the end, it further moves to Exit 2.

Meanwhile, Figure 11(b) shows the results when inference is performed using the sustained speed method. As observed in the figure, this method utilizes shallower exits even before thermal throttling occurs. This reduces the overall computational load on the system, leading to a slower

decline in CPU and GPU frequencies compared with the best-effort method. However, despite reducing computation, thermal throttling eventually occurs around 570 seconds, causing a drop in frequency. After this point, Phoenix compensates for latency constraints by selecting only Exit 1 and Exit 2 for inference. This experiment demonstrates that the best-effort method is well-suited for short-duration DNN inference tasks, whereas the sustained speed method provides better performance for prolonged execution.

Real-world Experiment. We conducted experiments to investigate Phoenix's capability to operate adaptively in real mobile execution environment. In real usage scenario, smartphones are typically held by people who input touches, and there may be background process during runtime. Figure 11(c) shows the results from an experiment where a Unity application, launched alongside a screen recorder, receives random touch interactions with best-effort method. First, as screen recorder requires GPU computing, 3D pose estimation model employs the first early-exit, right after the application launches. Second, due to the co-running screen recorder generating heat, CPU thermal throttling occurs at 150 seconds, earlier than previous cases. As a result, the face mesh task, which runs on the CPU, uses a shallower exit to meet latency constraints. Meanwhile, touch interactions cause frequency boosting, as seen in the fluctuation of CPU frequency trace. Due to enhanced computing capacity from frequency boosting, the face mesh model employs the exit 0 around 250 seconds. To sum up, Phoenix adaptively operates multi-instance multi-exit networks even in highly complex situations, achieving the target FPS and delivering its best effort.

Runtime Overhead. We conducted experiments to examine the runtime overhead of the proposed scheduler. The overhead primarily occurs during hardware setup and exit transitions. To quantify this, we measured the execution time of each function and calculated the proportion of spent time relative to the total execution time. The scheduler's overhead is approximately 0.1%, which is negligible. This is because hardware setup events are rare, and exit transitions, which occur infrequently between DNN executions, can be managed within 1 millisecond.

9 Related Works

9.1 Thermal Management Techniques

Many studies have explored solutions to keep devices within safe thermal limits by dynamically adjusting processor frequencies, using methods such as modeling-based method [38, 39], feedback control theory [40], and machine learning [11]. For example, zTT proposed a DRL-based algorithm that optimizes the operating frequency of both the CPU and GPU to manage thermal conditions. These DVFS-centric approaches are largely reactive, in that they regulate frequencies after thermal pressure emerges, and thus often struggle to sustain stable application-level latency/FPS once throttling persists. Consequently, they are limited in maintaining consistent FPS or user experience once throttling occurs. Complementary to DVFS control, THERMOS [41] explores thermal-aware RL scheduling on heterogeneous multi-chiplet PIM architectures, optimizing execution time–energy tradeoffs under thermal constraints. However, THERMOS does not address sustained mobile workloads where throttling progressively reduces compute availability, nor does it incorporate accuracy/latency adaptation via early-exit inference. In contrast, Phoenix proactively manages the thermal state by optimizing DNN task placement before heat accumulation reaches critical levels, and further mitigates post-throttling performance degradation through adaptive multi-exit inference.

9.2 Heterogeneous Mobile Processors

Heterogeneous mobile processors offer an opportunity to accelerate DNN inference on resource-constrained mobile devices through concurrent DNN execution. One approach is to accelerate a

single DNN inference by dividing the network into sub-graphs and executing these sub-graphs on different processors concurrently [38, 39]. However, these methods incur communication overhead while synchronizing the outputs from each processor. NN-stretch [42] was proposed to address this issue, yet it faces the limitation of not accounting for the varying computing capacities due to thermal throttling. Another approach is running multiple DNNs on multiple processors. Band [5] introduced a model analyzer and scheduler specifically designed to manage the workload of multiple DNNs on a heterogeneous mobile processor. However, this approach necessitates modifications to the code of deep learning frameworks, and it does not thoroughly consider thermal issues in the design of the inference system. Sung et al. [6] proposed a DAAS for scheduling multiple DNNs on mobile processors without framework modification. Yet, as our evaluation shows, their policy performs sub-optimally in sustained workloads where thermal throttling significantly alters device behavior. Surrogate-guided supervised learning methods [43, 44] have also been used for task allocation on heterogeneous mobile processors. However, under thermal throttling with multi-instance interference, surrogate-based labeling becomes unreliable for capturing long-term thermal effects, so we instead learn a policy via RL. Overall, existing heterogeneous scheduling methods optimize for latency or energy under the assumption of static processor performance. In contrast, Phoenix explicitly accounts for thermally-induced performance variations in multi-DNN settings, enabling sustainable long-term operation.

9.3 Multi-Exit Network

Recent works have attempted to mitigate performance degradation by designing multi-exit networks that consider the changing computing capacity according to the frequencies of mobile processors. Dělen [45] proposed an adaptive model-serving system for the on-device execution of multi-instance DNNs to maximize user experience. However, Dělen did not consider the neural architecture which can significantly impact the overall performance of on-device applications. Furthermore, Dělen's approach schedules tasks using only GPUs, thereby failing to leverage the advantages of heterogeneous mobile processors. Meanwhile, NAS has been extensively used to obtain optimal neural architectures. For instance, OmniLive [37] optimized a multi-exit network for video super resolution by exploring various early-exit networks based on the lowest and highest mobile GPU frequencies. HADAS [46] simultaneously optimized both the structure of the multi-exit network and the DVFS policy for energy-efficient DNN inference. However, to the best of our knowledge, none of the existing studies have thoroughly considered thermal throttling patterns in the design of neural architectures. Moreover, previous works often overlook the importance of selecting the appropriate number of exit networks, which can impact overall performance.

10 Discussion

We discuss issues related to the proposed approach and solutions. One may question whether this approach can be generalized to other types of devices and operating systems. Phoenix leverages learning-based solutions, NAS, and an RL-based scheduler to optimize the operation of multi-instance DNNs on mobile devices. Unlike rule-based solutions, learning-based solutions can adapt to unknown environments. For example, given a backbone network, NAS adjusts resource constraints specific to each device, enabling it to select the best early-exit networks. Next, recent efforts have explored RL-based scheduling and multi-exit network design to optimize inference systems. Phoenix differs in that it explicitly targets multi-instance real-time inference under thermal throttling, where compute availability varies over time and across processors. Phoenix provides an end-to-end scheme that jointly optimizes task allocation and adaptive early-exit inference using real-time workload and device-state observations to sustain FPS/QoE beyond throttling. To

our knowledge, Phoenix is among the first to couple thermal-aware task allocation with adaptive early-exit inference in an end-to-end manner for sustained real-time performance.

11 Conclusion

Video applications leveraging multiple concurrent DNNs are becoming increasingly common on mobile devices, making heat management critical for sustaining performance. In this article, we proposed Phoenix, the first thermal-aware multi-DNN inference system that proactively optimizes task allocation and network configuration to prevent thermal throttling. Unlike traditional reactive approaches based on DVFS, Phoenix anticipates thermal dynamics to maintain consistent inference quality over time. We plan to extend this work to running multiple DNNs on a wider range of devices, for instance, AR/VR goggles. These devices will execute various DNNs concurrently, including gaze tracking and hand pose estimation. By exploiting both the device and edge server, we expect that it will maximize the user experience of the target applications.

References

[1] T. Sawada, M. Katsurai, and M. Okubo. 2023. High-quality synthetic character image extraction via distortion recognition. In *Proceedings of the International Conference on Human-Computer Interaction*. Cope nhagen, Denmark, 23–28.

[2] J. Yi, S. Choi, and Y. Lee. 2020. EagleEye: Wearable camera-based person identification in crowded urban spaces. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, London, United Kingdom, 21–25.

[3] X. Zeng, B. Fang, H. Shen, and M. Zhang. 2020. Distream: Scaling live video analytics with workload-adaptive distributed edge intelligence. In *Proceedings of the 18th Conference on Embedded Networked Sensor Systems*, Yokohama, Japan, 16–19.

[4] J. Yi and Y. Lee. 2020. Heimdall: Mobile GPU coordination platform for augmented reality applications. In *Proceedings of the Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, London, United Kingdom. 21–25.

[5] J. S. J. Jeong, J. Lee, D. Kim, C. Jeon, C. Jeong, Y. Lee, and B.-G. Chun. 2022. Band: Coordinated multi-dnn inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, Portland, Oregon.

[6] H.-H. Sung, J.-A. Chen, W. Niu, J. Guan, B. Ren, and X. Shen. 2023. Decentralized application-level adaptive scheduling for multi-instance DNNs on open mobile devices. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC 23)*, Boston, MA, USA, 10–12

[7] B. Fang, X. Zeng, and M. Zhang. 2018. Nestdnn: Resource-aware multi-tenant on-device deep learning for continuous mobile vision. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking*, New Delhi, India.

[8] K. Sekar. 2013. Power and thermal challenges in mobile devices. In *Proceedings of the 19th Annual International Conference on Mobile Computing & Networking*, Miami, Florida, USA.

[9] Y. Hang and H. Kabban. 2015. Thermal management in mobile devices: Challenges and solutions. In *Proceedings of the 2015 31st Thermal Measurement, Modeling & Management Symposium (SEMI-THERM)*. IEEE, San Jose, CA, USA, 15–19.

[10] S. I. Venieris, I. Panopoulos, and I. S. Venieris. 2021. OODIn: An optimised on-device inference framework for heterogeneous mobile devices. In *Proceedings of the 2021 IEEE International Conference on Smart Computing (SMARTCOMP)*. IEEE, Irvine, CA, USA, 23–27.

[11] S. Kim, K. Bin, S. Ha, K. Lee, and S. Chong. ZTT: Learning-based DVFS with zero thermal throttling for mobile devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services, Virtual Conference*.

[12] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath. 2017. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine* 34, 6 (2017), 26–38.

[13] L. P. Kaelbling, M. L. Littman, and A. W. Moore. 1996. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research* 4 (1996), 237–285.

[14] S. Laskaridis, A. Kouris, and N. D. Lane. 2021. Adaptive inference through early-exit networks: Design, challenges and directions. In *Proceedings of the Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning, Virtual Conference*.

[15] S. Teerapittayanon, B. McDanel, and H.-T. Kung. 2016. Branchynet: Fast inference via early exiting from deep neural networks. In *Proceedings of the 2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE, Cancun, Mexico, 4–8.

[16] M. Almeida, S. Laskaridis, A. Mehrotra, L. Dudziak, I. Leontiadis, and N. D. Lane. 2021. Smart at what cost? characterising mobile deep neural networks in the wild. In *Proceedings of the 21st ACM Internet Measurement Conference, Virtual Conference*, 2–4.

[17] T. Benoit-Cattin, D. Velasco-Montero, and J. Fernández-Berni. 2020. Impact of thermal throttling on long-term visual inference in a CPU-based edge device. *Electronics* 9, 12 (2020), 2106.

[18] 2017. Google NNAPI. Retrieved from https://developer.android.com/ndk/guides/neuralnetworks

[19] Y.-H. Gong, J. J. Yoo, and S. W. Chung. 2017. Thermal modeling and validation of a real-world mobile ap. *IEEE Design & Test* 35, 1 (2017), 55–62.

[20] W. Jung, C. Kang, C. Yoon, D. Kim, and H. Cha. 2023. DevScope: A nonintrusive and online power analysis tool for smartphone hardware components. In *Proceedings of the Eighth IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis*, Tampere, Finland, 7–12.

[21] C. J. Watkins and P. Dayan. 1992. Q-learning. *Machine Learning* 8, 3 (1992), 279–292.

[22] J. Clifton and E. Laber. 2020. Q-learning: Theory and applications. *Annual Review of Statistics and Its Application* 7 (2020), 279–301.

[23] F. Tan, P. Yan, and X. Guan. 2017. Deep reinforcement learning: from Q-learning to deep Q-learning. In *Proceedings of the Neural Information Processing: 24th International Conference, ICONIP 2017*, Guangzhou, China.

[24] J. Fan, Z. Wang, Y. Xie, and Z. Yang. 2020. A theoretical analysis of deep Q-learning. In *Proceedings of the Learning for Dynamics and Control. PMLR, Virtual Conference*, 13–18.

[25] A. Kouris, S. I. Venieris, S. Laskaridis, and N. D. Lane. 2022. Adaptable mobile vision systems through multi-exit neural networks. In *Proceedings of the Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. Portland, Oregon, USA.

[26] S. Laskaridis, S. I. Venieris, H. Kim, and N. D. Lane. 2020. HAPI: Hardware-aware progressive inference. In *Proceedings of the 39th International Conference on Computer-Aided Design. Virtual Conference*.

[27] T. Elsken, J. H. Metzen, and F. Hutter. 2019. Neural architecture search: A survey. *The Journal of Machine Learning Research* 20, 1 (2019), 1997–2017.

[28] M. Wistuba, A. Rawat, and T. Pedapati. 2019. A survey on neural architecture search. *arXiv preprint arXiv:1905.01392*.

[29] A. Bakhtiarnia, Q. Zhang, and A. Iosifidis. 2021. Improving the accuracy of early exits in multi-exit architectures via curriculum learning. In *Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, Shenzhen, China, 18–22.

[30] VTUBER (Virtual Yotuber) Market size, share, growth, and industry analysis, by type (2D VTuber, 3D VTuber), by application (livestreaming & performance, digital contents & derivative, others) and regional insight and forecast to 2031. 2025. Retrieved from https://www.businessresearchinsights.com/market-reports/VTuber-virtual-youtuber-market-109503

[31] S. Choi, S. Choi, and C. Kim. 2021. MobileHumanPose: Toward real-time 3D human pose estimation in mobile devices. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Virtual Conference*, 19–25.

[32] MediaPipe Face Detection. 2025. *Retrieved from* https://github.com/google/mediapipe/blob/master/docs/solutions/face_detection.md

[33] S. Chen, Y. Liu, X. Gao, and Z. Han. 2018. Mobilefacenets: Efficient cnns for accurate real-time face verification on mobile devices. In *Proceedings of the Biometric Recognition: 13th Chinese Conference, CCBR 2018*, Urumqi, China, 11–12.

[34] MediaPipe Tasks Face Landmark Detection. 2025. Retrieved from https://github.com/googlesamples/mediapipe/tree/main/examples/face_landmarker/android

[35] 2025. Jocher G. Yolov5. Retrieved from https://github.com/ultralytics/yolov5

[36] M. Tan and Q. Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning. PMLR*, Long Beach, California, USA

[37] S. Park, Y., H. Jun, J. Lee, and H. Cha. 2023. Omnilive: Super-resolution enhanced 360 video live streaming for mobile devices. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, Helsinki, Finland, 18–22.

[38] G. Bhat, G. Singla, A. K. Unver, and U. Y. Ogras. 2017. Algorithmic optimization of thermal and power management for heterogeneous mobile platforms. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 26, 3 (2017), 544–557.

[39] S. Wang, G. Ananthanarayanan, and T. Mitra. 2018. OPTiC: Optimizing collaborative CPU–GPU computing on mobile devices with thermal constraints. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 3 (2018), 393–406.

[40] O. Sahin, L. Thiele, and A. K. Coskun. 2018. MAESTRO: Autonomous QoS management for mobile applications under thermal constraints. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 8 (2018), 1557–1570.

[41] Kanani Alish et al. 2025. THERMOS: Thermally-Aware multi-objective scheduling of AI workloads on heterogeneous multi-chiplet PIM architectures. *ACM Transactions on Embedded Computing Systems* 24, 5s (2025), 1–26.

[42] Sumit K. Mandal et al. 2020. An energy-aware online learning framework for resource management in heterogeneous platforms. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 25, 3 (2020), 1–26.

[43] Gaurav Narang et al. 2023. Uncertainty-aware online learning for dynamic power management in large manycore systems. *2023 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE.

[44] J. Wei, T. Cao, S. Cao, S. Jiang, S. Fu, M. Yang, Y. Zhang, and Y. Liu. 2023. NN-Stretch: Automatic neural network branching for parallel inference on heterogeneous multi-processors. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, Helsinki, Finland.

[45] Q. Liang, W. A. Hanafy, N. Bashir, A. Ali-Eldin, D. Irwin, and P. Shenoy. 2023. Dělen: enabling flexible and adaptive model-serving for multi-tenant edge AI. In *Proceedings of the 8th ACM/IEEE Conference on Internet of Things Design and Implementation*, SanAntonio, Texas, USA, 9–12.

[46] H. Bouzidi, M. Odema, H. Ouarnoughi, M. A. Al Faruque, and S. Niar. 2023. HADAS: Hardware-aware dynamic neural architecture search for edge performance scaling. In *Proceedings of the 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Antwerp, Belgium, 17–19.

Received 13 May 2025; revised 1 January 2026; accepted 20 January 2026